



```
In [1]: # Numpy Crash Course
```

```
In [2]: import numpy as np
```

```
In [3]: np.__version__
```

```
Out[3]: '2.3.5'
```

```
In [4]: import sys  
sys.version
```

```
Out[4]: '3.13.9 | packaged by Anaconda, Inc. | (main, Oct 21 2025, 19:09:58) [MSC v.1  
929 64 bit (AMD64)]'
```

```
In [5]: ## Creating Arrays
```

```
In [6]: my_list = [0,1,2,3,4,5]  
my_list
```

```
Out[6]: [0, 1, 2, 3, 4, 5]
```

```
In [7]: type(my_list)
```

```
Out[7]: list
```

```
In [8]: arr = np.array(my_list)
```

```
In [9]: arr
```

```
Out[9]: array([0, 1, 2, 3, 4, 5])
```

```
In [10]: type(arr)
```

```
Out[10]: numpy.ndarray
```

```
In [11]: type(my_list)
```

```
Out[11]: list
```

```
In [12]: # numpy arrange
```

```
In [13]: np. # we learn important function
```

```
Cell In[13], line 1  
    np. # we learn important function  
      ^  
SyntaxError: invalid syntax
```

```
In [ ]: np.arange(15)
```

```
In [ ]: np.arange(3.0)
```

```
In [ ]: np.arange(10)
```

```
In [ ]: np.arange(0, 5)
```

```
In [ ]: np.arange(10,20)
```

```
In [ ]: np.arange(20,10) # 1st arg < 2nd arg
```

```
In [ ]: np.arange(-20,10)
```

```
In [ ]: np.arange(-16,10)
```

```
In [ ]: np.arange(-20,-10)
```

```
In [ ]: np.arange(30,20) # 1st arg always be < then 2nd arg
```

```
In [ ]: np.arange()
```

```
In [ ]: np.arange(10,30,5) # 10- starting from 30- end point 5 - step count
```

```
In [ ]: np.arange(0,10,3)
```

```
In [ ]: np.arange(10,30,5,8)
```

```
In [ ]: np.zeros(3) # parameter tuning
```

```
In [ ]: np.zeros(5, dtype=int) # hyperparameter tuning
```

```
In [ ]: np.zeros((2,2), dtype=int)
```

```
In [ ]: zero = np.zeros([2,2])  
print(zero)  
print(type(zero))
```

```
In [ ]: zero = np.zeros([2,2])  
print(zero)  
  
print('####')  
  
print(type(zero))
```

```
In [ ]: np.zeros((2,10))
```

```
In [ ]: np.zeros((2,2))
```

```
In [ ]: np.zeros((3,3))
```

```
In [ ]: np.zeros((10,30))
```

```
In [ ]: np.zeros((5,10)) # by default large -- will give row & 2nd arg - columns
```

```
In [ ]: n = (6,7)
n1 = (6,8)
print(np.zeros(n1)) # parameter tuning
```

```
In [ ]: print(np.zeros(n,dtype=int)) ## hyperparameter tuning
```

```
In [ ]: n
```

```
In [ ]: n1
```

```
In [ ]: print(np.zeros(n1))
```

```
In [ ]: # numpy with ones
```

```
In [ ]: np.ones(3)
```

```
In [ ]: np.ones(4, dtype=int)
```

```
In [ ]: np.ones(4)
```

```
In [ ]: n
```

```
In [ ]: np.ones(n)
```

```
In [ ]: np.ones((5,4),dtype=int) # by default 5- rows & 4 - columns
```

```
In [ ]: np.
```

```
In [ ]: np.twos((2,3))
```

```
In [ ]: np.three(2,3)
```

```
In [ ]: np.ones(2)
```

```
In [ ]: np.ones((2,4))
```

```
In [ ]: np.ones((6,10),dtype = int)
```

```
In [ ]: np.twos((2,4))
```

```
In [ ]: np.
```

```
In [ ]: np.three((2,4))
```

```
In [ ]: range(5)
```

```
In [ ]: r = range(5)  
r
```

```
In [ ]: for i in r:  
        print(i)
```

```
In [ ]: list(range(5))
```

```
In [ ]: range(1,10)
```

```
In [ ]: list(range(1,10))
```

```
In [ ]: list(range(1,10,3))
```

```
In [ ]: y = list(range(12))  
y
```

```
In [ ]: # Random value
```

```
In [ ]: rand(3,2)
```

```
In [ ]: rand(3,2)  
random.rand(3,2)
```

```
In [ ]: np.random.rand(5)
```

```
In [ ]: np.rand(4)
```

```
In [ ]: np.random.rand(2,4)
```

```
In [ ]: np.random.randint(2,4)
```

```
In [ ]: np.random.randint(2,20) # 2nd argument is exclusive
```

```
In [ ]: np.random.randint(0,1)
```

```
In [ ]: np.random.randint(10,20,5)
```

```
In [ ]: np.random.randint(1,6,4)
```

```
In [ ]: np.random.rand(3)
```

```
In [ ]: np.random.randint(1)
```

```
In [ ]: np.random.randint(30,20,10)
```

```
In [ ]: np.random.randint(-30,20,10)
```

```
In [ ]: np.random.randint(20,30,10)
```

```
In [ ]: np.random.randint(5,9)  #GET THE VALUE <=1 & >=5
```

```
In [ ]: np.random.randint(10,21,3)
```

```
In [ ]: np.random.randint(1,12,10)
```

```
In [ ]: np.random.randint(10,40,(10,10)) #generre the element 10 -30 with 4*4 mtri
```

```
In [ ]: np.random.randint(1,100,(12,12)) #generre the element 10 -30 with 4*4 mtri
```

```
In [ ]: np.arange(1,13).reshape(3,4)
```

```
In [ ]: np.arange(1,13).reshape(12, 1)
```

```
In [ ]: b = np.random.randint(10,20,(5,4))  
b
```

```
In [ ]: type(b)
```

```
In [ ]: b
```

```
In [ ]: b[:]
```

```
In [ ]: b[1:3]
```

```
In [ ]: b
```

```
In [ ]: b[1,2]
```

```
In [ ]: b
```

```
In [ ]: b[1,3]
```

```
In [ ]: b[1,-1]
```

```
In [ ]: b
```

```
In [ ]: b[2:3]
```

```
In [ ]: b
```

```
In [ ]: b[0:-2]
```

```
In [ ]: b
```

```
In [ ]: b[-5,-3]
```

```
In [ ]: b
```

```
In [ ]: b[-4,2]
```

```
In [ ]: np.random.randint(10,20,(4,4))
```

```
In [ ]: b
```

```
In [ ]: b[-4,-2]
```

```
In [ ]: b
```

```
In [ ]: b[-4:2]
```

```
In [ ]: b[:]
```

```
In [ ]: ## Operations
```

```
In [ ]: a = np.random.randint(10,20,10)  
a
```

```
In [ ]: id(a)
```

```
In [ ]: arr
```

```
In [ ]: arr2 = np.random.randint(0,100,(10,10))
```

```
In [ ]: arr2
```

```
In [ ]: arr
```

```
In [ ]: arr[:]
```

```
In [ ]: arr
```

```
In [ ]: arr[:4]
```

```
In [ ]: arr2[:]
```

```
In [ ]: arr2[0:5]
```

```
In [ ]: arr2
```

```
In [ ]: arr2[1,4]
```

```
In [ ]: arr2
```

```
In [ ]: arr2[-5,5]
```

```
In [ ]: arr2[-5,-5]
```

```
In [ ]: arr2
```

```
In [ ]: arr2[-5,-5]
```

```
In [ ]: arr2
```

```
In [ ]: arr2[-1,-2]
```

```
In [ ]: arr2
```

```
In [ ]: arr2[::-1]
```

```
In [ ]: arr2[::-1]
```

```
In [ ]: arr2
```

```
In [ ]: arr2[::-2]
```

```
In [ ]: arr2
```

```
In [ ]: arr2[::-3]
```

```
In [ ]: arr2
```

```
In [ ]: arr2[: -3]
```

```
In [ ]: arr
```

```
In [ ]: arr.max()
```

```
In [ ]: arr.min()
```

```
In [ ]: arr
```

```
In [ ]: arr.mean()
```

```
In [ ]: arr
```

```
In [ ]: arr.median()
```

```
In [ ]: from numpy import *  
a = array([1,2,3,4,9])  
median(a)
```

```
In [ ]: Without work on import* can you please find the median, mode)
```

```
In [ ]: arr
```

```
In [ ]: arr.reshape(3,2)
```

```
In [ ]: arr.reshape(6,1)
```

```
In [ ]: arr.reshape(1,6)
```

```
In [ ]: arr
```

```
In [ ]: arr.reshape(2,4)
```

```
In [ ]: arr
```

```
In [ ]: arr.reshape(2,3,order='C')
```

```
In [ ]: arr.reshape(2,3,order='F') # print element with fortran
```

```
In [ ]: arr.reshape(2,3,order='A') # A almost give you c type output
```

```
In [ ]: arr
```

```
In [ ]: arr.reshape(2,3)
```

```
In [ ]: arr.reshape(1,4)
```

```
In [ ]: arr.reshape(1,6)
```

```
In [ ]: arr.reshape(6,1)
```

```
In [ ]: arr.reshape(2,6)
```

```
In [ ]: arr.reshape(3,3)
```

```
In [ ]: arr
```

```
In [ ]: arr.reshape(3,2)
```

```
In [ ]: ## Indexing
```

```
In [ ]: mat = np.arange(0,100).reshape(10,10)
```



```
mat
```

```
In [ ]: row = 4  
col = 5
```

```
In [ ]: col
```

```
In [ ]: row
```

```
In [ ]: mat
```

```
In [ ]: mat[6] # default it represent to rows
```

```
In [ ]: mat
```

```
In [ ]: # With Slices  
mat[:,col]
```

```
In [ ]: mat
```

```
In [ ]: mat[row,:]
```

```
In [ ]: mat
```

```
In [ ]: mat
```

```
In [ ]: mat[:]
```

```
In [ ]: mat[:,8]
```

```
In [ ]: mat
```

```
In [ ]: mat[:, -1]
```

```
In [ ]: mat
```

```
In [ ]: row
```

```
In [ ]: col
```

```
In [ ]: mat[:,col]
```

```
In [ ]: mat
```

```
In [ ]: mat[1,4]
```

```
In [ ]: mat
```

```
In [ ]: mat[1:4]
```

```
In [ ]: mat[3:-3]
```

```
In [ ]: mat[0]
```

```
In [ ]: mat[6]
```

```
In [ ]: mat
```

```
In [ ]: mat[6:]
```

```
In [ ]: mat[:6]
```

```
In [ ]: mat[5:7]
```

```
In [ ]: mat[0:10]
```

```
In [ ]: mat[0:10:3]
```

```
In [ ]: mat[0:10:3]
```

```
In [ ]: mat[4:]
```

```
In [ ]: mat[::-1]
```

```
In [ ]: mat[::-3]
```

```
In [ ]: mat[::-3]
```

```
In [ ]: mat[::-5]
```

```
In [ ]: mat[2:6]
```

```
In [ ]: mat[2:6,2:4] # 1:5 --> only row part /// 1:3 -- it indicates only column parts
```

```
In [ ]: mat[0,1]
```

```
In [ ]: mat[1,6]
```

```
In [ ]: mat[1:]
```

```
In [ ]: mat
```

```
In [ ]: mat[:6]
```

```
In [ ]: mat[0:1]
```

```
In [ ]: mat[1:2,2:4]
```

```
In [ ]: mat[2:3,2:3]
```

```
In [ ]: mat[3:5,2:4]
```

```
In [ ]: mat[2:3,4:5]
```

```
In [ ]: ## Masking
```

```
In [ ]: mat # we also called as filter
```

```
In [ ]: id(mat)
```

```
In [ ]: mat
```

```
In [ ]: mat[mat<50]
```

```
In [ ]: mat > 50
```

```
In [ ]: mat[mat==50]
```

```
In [ ]: mat
```

```
In [ ]: mat == 50
```

```
In [ ]: a1 = mat[mat<50]  
a1
```

```
In [ ]: a2 = mat[mat>50]  
a2
```

```
In [ ]: a3 = mat[mat<=50]  
a3
```

```
In [ ]: a4 = mat[mat==50]  
a4
```

```
In [ ]: # python program to generat otp
```

```
In [ ]: import random  
  
def generate_otp(length=4):  
    """Generate a numeric OTP of a specified length."""  
    digits = '012345'  
    otp = ''.join(random.choice(digits) for _ in range(length))  
    return otp
```

```
# Example usage
otp_length = 4 # You can change this to any length you prefer
otp = generate_otp(otp_length)
print(f"Your OTP is: {otp}")
```

```
In [ ]: def wish():
        print('good even')
        wish()

        def wish():
            print('good even')
            wish()

        def wish():
            print('good even')
            wish()
```

```
In [ ]: list1=['a','b','g',1,5]
        print(list1.pop)
```

```
In [ ]: x = [1, 2, 3]
        y = x.copy()
        x.append(4)
        print(x)
```

```
In [ ]: ## Creating Arrays
```

```
In [ ]: my_list = [0,1,2,3,4,5]
        my_list
```

```
In [ ]: type(my_list)
```

```
In [ ]: arr = np.array(my_list)
```

```
In [ ]: arr
```

```
In [ ]: type(arr)
```

```
In [ ]: type(my_list)
```

```
In [ ]: np. # we learn important function
```

```
In [ ]: np.arange(15)
```

```
In [ ]: np.arange(3.0)
```

```
In [ ]: np.arange(10)
```

```
In [ ]: np.arange(0, 5)
```

```
In [ ]: np.arange(10,20)
```

```
In [ ]: np.arange(20,10) # 1st arg < 2nd arg
```

```
In [ ]: np.arange(-20,10)
```

```
In [ ]: np.arange(-16,10)
```

```
In [ ]: np.arange(-20,-10)
```

```
In [ ]: np.arange(30,20) # 1st arg always be < then 2nd arg
```

```
In [ ]: ar = np.arange(-30,20)
ar
```

```
In [ ]: np.arange(10,10)
```

```
In [ ]: np.arange()
```

```
In [ ]: np.arange(10,30,5) # 10- starting from 30- end point 5 - step count
```

```
In [ ]: np.arange(0,10,3)
```

```
In [ ]: np.arange(10,30,5,8)
```

```
In [ ]: np.zeros(3) # parameter tuning
```

```
In [ ]: np.zeros(5, dtype=int) # hyperparameter tuning
```

```
In [ ]: np.zeros((2,2), dtype=int)
```

```
In [ ]: zero = np.zeros([2,2])
print(zero)
print(type(zero))
```

```
In [ ]: zero = np.zeros([2,2])
print(zero)

print('####')

print(type(zero))
```

```
In [ ]: np.zeros((2,10))
```

```
In [ ]: np.zeros((2,2))
```

```
In [ ]: np.zeros((3,3))
```

```
In [ ]: np.zeros((10,30))
```

```
In [ ]: np.zeros((5,10)) # by default large -- will give row & 2nd arg - columns
```

```
In [ ]: n = (6,7)
n1 = (6,8)
print(np.zeros(n1)) # parameter tuning
```

```
In [ ]: print(np.zeros(n1)) # parameter tuning
```

```
In [ ]: n
```

```
In [ ]: n1
```

```
In [ ]: print(np.zeros(n1))
```

```
In [ ]: np.ones(3)
```

```
In [ ]: np.ones(4, dtype=int)
```

```
In [ ]: np.ones(4)
```

```
In [ ]: n
```

```
In [ ]: np.ones(n)
```

```
In [ ]: np.ones((5,4), dtype=int) # by default 5- rows & 4 - columns
```

```
In [ ]: np.
```

```
In [ ]: np.twos((2,3))
```

```
In [ ]: np.three(2,3)
```

```
In [ ]: np.ones(2)
```

```
In [ ]: np.ones((2,4))
```

```
In [ ]: np.ones((6,10), dtype = int)
```

```
In [ ]: np.ones((6,10), dtype = int)
```

```
In [ ]: np.
```

```
In [ ]: np.three((2,4))
```

```
In [ ]: range(5)
```

```
In [ ]: r = range(5)
        r
```

```
In [ ]: for i in r:
        print(i)
```

```
In [ ]: list(range(5))
```

```
In [ ]: list(range(1,10))
```

```
In [ ]: list(range(1,10,3))
```

```
In [ ]: y = list(range(12))
        y
```

```
In [ ]: rand(3,2)
```

```
In [ ]: rand(3,2)
        random.rand(3,2)
```

```
In [ ]: np.random.rand(5)
```

```
In [ ]: np.rand(4)
```

```
In [ ]: np.random.rand(2,4)
```

```
In [ ]: np.random.rand(2,4)
```

```
In [ ]: np.random.randint(2,20) # 2nd argument is exclusive
```

```
In [ ]: np.random.randint(0,1)
```

```
In [ ]: np.random.randint(0,1)
```

```
In [ ]: np.random.randint(10,20,5)
```

```
In [ ]: np.random.randint(1,6,4)
```

```
In [ ]: np.random.rand(3)
```

```
In [ ]: np.random.randint(1)
```

```
In [ ]: np.random.randint(30,20,10)
```

```
In [ ]: np.random.randint(-30,20,10)
```

```
In [ ]: np.random.randint(20,30,10)
```

```
In [ ]: np.random.randint(5,9)  #GET THE VALUE <=1 & >=5
```

```
In [ ]: np.random.randint(10,21,3)
```

```
In [ ]: np.random.randint(1,12,10)
```

```
In [ ]: np.random.randint(10,40,(10,10)) #generre the element 10 -30 with 4*4 mtri
```

```
In [ ]: np.random.randint(1,100,(12,12)) #generre the element 10 -30 with 4*4 mtri
```

```
In [ ]: np.arange(1,13).reshape(3,4)
```

```
In [ ]: np.arange(1,13).reshape(12, 1)
```

```
In [ ]: b = np.random.randint(10,20,(5,4))  
b
```

```
In [ ]: type(b)
```

```
In [ ]: b
```

```
In [ ]: b[:,]
```

```
In [ ]: b[1:3]
```

```
In [ ]: b
```

```
In [ ]: b[1,2]
```

```
In [ ]: b
```

```
In [ ]: b[1,3]
```

```
In [ ]: b[1,-1]
```

```
In [ ]: b
```

```
In [ ]: b[2:3]
```

```
In [ ]: b
```

```
In [ ]: b[0:-2]
```

```
In [ ]: b
```

```
In [ ]: b[0,2]
```



```
In [ ]: b
```

```
In [ ]: b[-5,-3]
```

```
In [ ]: b
```

```
In [ ]: b[-4,2]
```

```
In [ ]: np.random.randint(10,20,(4,4))
```

```
In [ ]: b
```

```
In [ ]: b[-4,-2]
```

```
In [ ]: b[-4:2]
```

```
In [ ]: b
```

```
In [ ]: b[:]
```

```
In [ ]: ## Operations
```

```
In [ ]: a = np.random.randint(10,20,10)  
a
```

```
In [ ]: id(a)
```

```
In [ ]: arr
```

```
In [ ]: arr2 = np.random.randint(0,100,(10,10))
```

```
In [ ]: arr2
```

```
In [ ]: arr
```

```
In [ ]: arr[:]
```

```
In [ ]: arr
```

```
In [ ]: arr[:4]
```

```
In [ ]: arr2[:]
```

```
In [ ]: arr2[0:5]
```

```
In [ ]: arr2
```

```
In [ ]: arr2[1,4]
```

```
In [ ]: arr2
```

```
In [ ]: arr2[-5,5]
```

```
In [ ]: arr2[-5,-5]
```

```
In [ ]: arr2
```

```
In [ ]: arr2[-5,-5]
```

```
In [ ]: arr2
```

```
In [ ]: arr2[-1,-2]
```

```
In [ ]: arr2
```

```
In [ ]: arr2[::-1]
```

```
In [ ]: arr2
```

```
In [ ]: arr2[::-2]
```

```
In [ ]: arr2[::-2]
```

```
In [ ]: arr2
```

```
In [ ]: arr2[::-3]
```

```
In [ ]: arr2
```

```
In [ ]: arr2[: -3]
```

```
In [ ]: arr
```

```
In [ ]: arr.min()
```

```
In [ ]: arr
```

```
In [ ]: arr.mean()
```

```
In [ ]: arr
```

```
In [ ]: arr.median()
```

```
In [ ]: from numpy import *
```

```
a = array([1,2,3,4,9])
median(a)
```

```
In [ ]: # Without work on import* can you please find the median, mode)
```

```
In [ ]: arr
```

```
In [ ]: arr.reshape(3,2)
```

```
In [ ]: arr.reshape(6,1)
```

```
In [ ]: arr.reshape(1,6)
```

```
In [ ]: arr
```

```
In [ ]: arr.reshape(2,4)
```

```
In [ ]: arr
```

```
In [ ]: arr.reshape(2,3,order='C')
```

```
In [ ]: arr.reshape(2,3,order='F') # print element with fortran
```

```
In [ ]: arr.reshape(2,3,order='A') # A almost give you c type output
```

```
In [ ]: arr
```

```
In [ ]: arr.reshape(2,3)
```

```
In [ ]: arr.reshape(1,4)
```

```
In [ ]: arr.reshape(1,6)
```

```
In [ ]: arr.reshape(2,6)
```

```
In [ ]: arr.reshape(3,3)
```

```
In [ ]: arr
```

```
In [ ]: arr.reshape(3,2)
```

```
In [15]: import matplotlib.pyplot as plt
```

```
In [16]: from PIL import Image
```

```
In [17]: butt_image=Image.open(r'C:\Users\HP\OneDrive\Pictures\butt.jpeg')
```

```
In [18]: butt_image
```

```
Out[18]:
```



```
In [20]: type(butt_image)
```

```
Out[20]: PIL.JpegImagePlugin.JpegImageFile
```

```
In [21]: butt_arr=np.asarray(butt_image)  
butt_arr
```

```

Out[21]: array([[[ 0,  4,  5],
                  [ 0,  4,  5],
                  [ 0,  4,  5],
                  ...,
                  [12, 28, 18],
                  [12, 28, 18],
                  [12, 28, 18]],

                [[ 0,  4,  5],
                  [ 0,  4,  5],
                  [ 0,  4,  5],
                  ...,
                  [14, 30, 20],
                  [14, 30, 20],
                  [14, 30, 20]],

                [[ 0,  3,  4],
                  [ 0,  3,  4],
                  [ 0,  3,  4],
                  ...,
                  [14, 30, 20],
                  [14, 30, 20],
                  [14, 30, 20]],

                ...,

                [[11, 22, 18],
                  [ 4, 15, 11],
                  [ 6, 17, 13],
                  ...,
                  [15, 25, 17],
                  [15, 25, 17],
                  [15, 25, 17]],

                [[13, 24, 20],
                  [ 5, 16, 12],
                  [ 6, 17, 13],
                  ...,
                  [15, 25, 17],
                  [15, 25, 17],
                  [15, 25, 17]],

                [[14, 25, 21],
                  [ 6, 17, 13],
                  [ 7, 18, 14],
                  ...,
                  [15, 25, 17],
                  [15, 25, 17],
                  [15, 25, 17]]], shape=(116, 175, 3), dtype=uint8)

```

```
In [22]: plt.imshow(butt_image)
```

```
Out[22]: <matplotlib.image.AxesImage at 0x2292b383cb0>
```



```
In [23]: butt_arr.shape
```

```
Out[23]: (116, 175, 3)
```